Laborator 1

Noțiuni de bază de programare. Limbajul C

Introducere

Programarea calculatoarelor este procesul de proiectare, implementare, testare și depanare a codului sursă al programelor de calculator. Codul sursă este scris într-un limbaj de programare și poate reprezenta fie o modificare a unui cod existent, fie ceva complet nou. Scopul programării este de a crea programe care să se comporte într-un mod specific. Scrierea codului sursă necesită, de obicei, cunoștințe aprofundate în diverse domenii, cum ar fi logica formală și algoritmii.

Un algoritm este o succesiune de pași ce pot fi aplicați mecanic pentru a rezolva o anumită clasă de probleme. Pentru scrierea unui algoritm, se pot folosi următoarele metode:

- **Scheme logice**
- **Pseudocod** – o formă intermediară de reprezentare a algoritmului, menită să faciliteze trecerea la un limbaj de programare și să ofere claritate rapid.
- **Mental**, fără pseudocod sau scheme logice.

Cerințe pentru un algoritm:

- **Claritate** – algoritmul nu trebuie să aibă ambiguități.
- **Generalitate** – trebuie să rezolve o clasă întreagă de probleme, nu doar cazuri particulare.
- **Finitudine** – algoritmul trebuie să ofere un rezultat într-un timp finit.

Elementele de bază ale algoritmilor:

- **Date**: de intrare și de ieșire.
- **Tipuri de date**: întregi, reale, logice (0/1 sau False/True), caractere (ex. ‘a’), șiruri de caractere (ex. “abc”).
- **Constante**: date din program care nu trebuie modificate sau calculate (ex. PI, e).
- **Variabile**: au un nume unic, dar valoarea poate varia; caracteristici importante: nume, tip, valoare la un moment dat, adresă în memorie. Exemplu: x este de tip întreg, are valoarea 10 și se află la adresa de memorie 0xFF38.
- **Expresii**: construite cu ajutorul constantelor, variabilelor și operatorilor; pot avea tipuri diferite, la fel ca variabilele. Exemplu: 10*x+2; x>y; x+y.
- **Operații**:
    - **Intrare**: preluarea datelor de la un dispozitiv de intrare.

- **Ieșire**: transmiterea datelor din memorie către un dispozitiv de ieșire.
- **Decizie și atribuirea**: ex. x = 7; y = x + y (se evaluează expresia din partea dreaptă, iar valoarea este atribuită variabilei din partea stângă, care își pierde valoarea anterioară).

Un **program** reprezintă o descriere precisă și concisă a unui algoritm, scris într-un anumit limbaj de programare.

Tipuri de limbaje de programare:

- **Limbaje de nivel scăzut**, dependente de calculator (ex. limbaj de asamblare, care folosește mnemonice pentru operații și simboluri pentru adrese; este greu de programat și specific unei mașini).
- **Limbaje de nivel înalt**, independente de arhitectura hardware a calculatorului și sunt mai aproape de limbajul natural uman, ceea ce le face mai ușor de învățat și utilizat. Aceste limbaje permit programatorilor să scrie cod fără să se preocupe de detaliile tehnice legate de gestionarea memoriei sau de comunicația directă cu hardware-ul (ex. Java, C, C++, Python, C#, PHP, etc).

Programarea structurată este o metodă de dezvoltare a software-ului care se bazează pe utilizarea unor structuri logice clare și bine definite pentru organizarea codului. Aceasta a fost introdusă în anii 1970 și reprezintă o alternativă la programarea nestructurată, care tindea să fie dezordonată și greu de întreținut.

Principalele caracteristici ale programării structurate sunt:

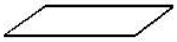
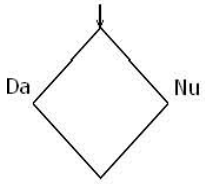
1. **Abordarea „top-down”**: Problemele complexe sunt descompuse în subprobleme mai mici, numite **module** sau **subrutine**, care pot fi dezvoltate și testate independent. Această metodă face programul mai ușor de înțeles și de întreținut.
2. **Utilizarea a trei structuri fundamentale de control**:
   - **Secvență**: Instrucțiunile sunt executate una după alta, în ordine strictă.
   - **Decizie** (sau alternativă): Permite ramificarea logicii programului prin intermediul condițiilor (ex. instrucțiunile if-else).
   - **Repetiție** (sau ciclu): Permite execuția repetată a unui bloc de cod, până când o anumită condiție este îndeplinită (ex. bucle while sau for).
3. **Fără utilizarea instrucțiunilor de tip „go to”**: Programarea structurată evită instrucțiunile „go to”, care transferă necontrolat execuția programului la o altă linie de cod, deoarece acestea fac codul dificil de urmărit și înțeles. În locul lor, se folosesc structurile controlate, cum ar fi buclele și deciziile.
4. **Modularitate**: Codul este împărțit în **module** independente, fiecare având o funcție bine definită. Acest lucru facilitează înțelegerea și testarea programului, precum și reutilizarea codului în alte proiecte.

Avantajele programării structurate includ:

- Cod mai **clar**, ușor de citit și de înțeles.
- **Întreținere** mai simplă, datorită organizării clare și a modularității.
- **Depanare** și testare mai eficientă, deoarece modulele pot fi testate individual.
- **Reutilizarea codului** – modulele scrise pentru o aplicație pot fi folosite și în alte aplicații.

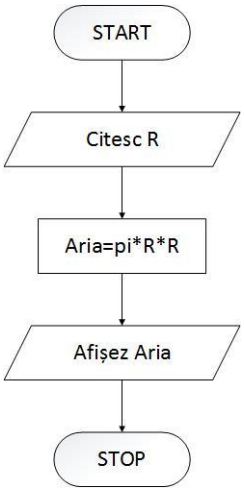
Scheme logice

Schemele logice sunt o "reprezentare" schematică a unui program. Ele sunt valabile în orice limbaj de programare, folosirea lor permițând o dezvoltare și o vizualizarea mai ușoară a programului. O *schemă logic*ă are în componență următoarele blocuri:

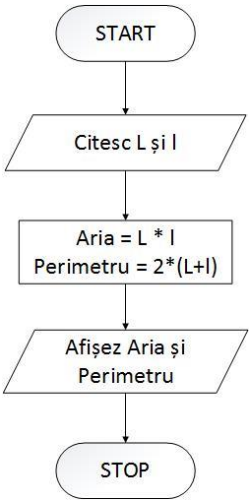
	Bloc de început	Marchează începutul algoritmului
	Bloc de sfârşit	Marchează sfârşitul algoritmului
	Bloc de intrare-ieşire	Realizează citirea datelor, respectiv afişarea rezultatelor
	Bloc de calcul	Efectuează un calcul, sau o atribuire
	Bloc decizional	Implică luarea unei decizii. În cazul în care condiţia înscrisă în interiorul blocului este satisfăcută atunci algoritmul se va înscrie pe ramura DA, iar în caz contrar, pe ramura NU
	Arc orientat	Descrie sensul şi direcţia parcurgerii algoritmului

Probleme rezolvate cu scheme logice

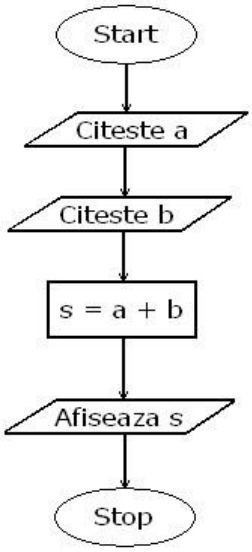
1. Se citește de la tastatură raza unui cerc. Să se calculeze aria cercului.



2. Se citește de la tastatură lungimea L și lățimea l a unui dreptunghi. Să se calculeze Aria și Perimetrul dreptunghiului.

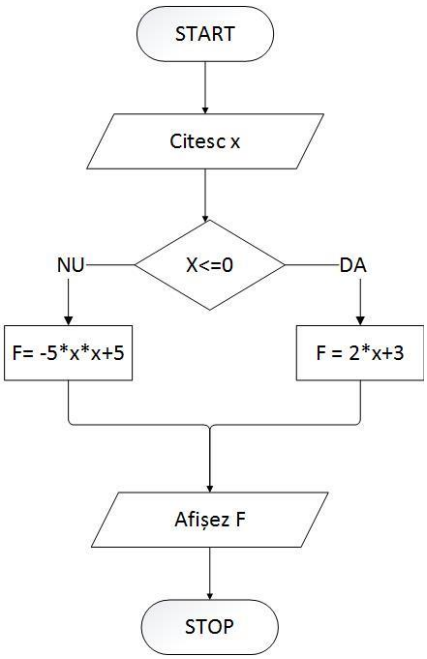


3. Se citesc de la tastatură două numere, să se calculeze suma lor.

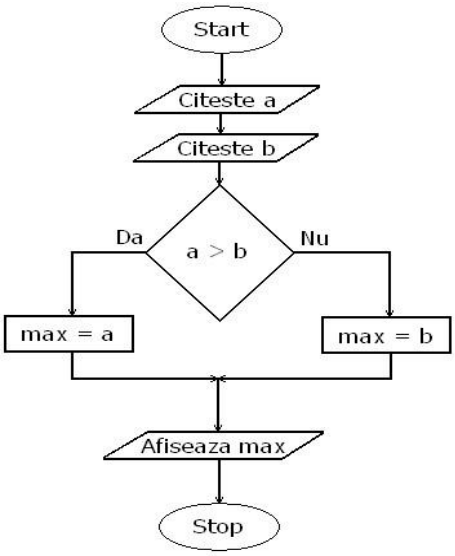


4. Să se implementeze următoarea expresie:

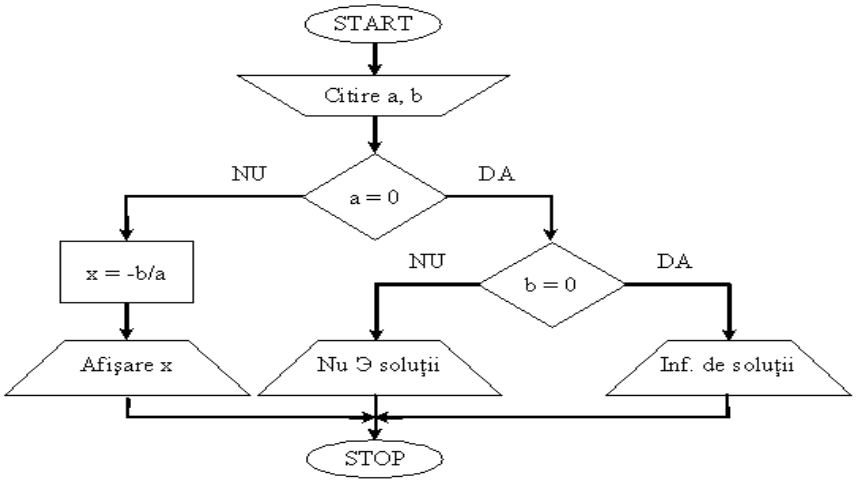
$$F = \begin{cases} 2 * x + 3\,, & x \leq 0 \\ -5 * x^2 + 5\,, & x > 0 \end{cases}$$



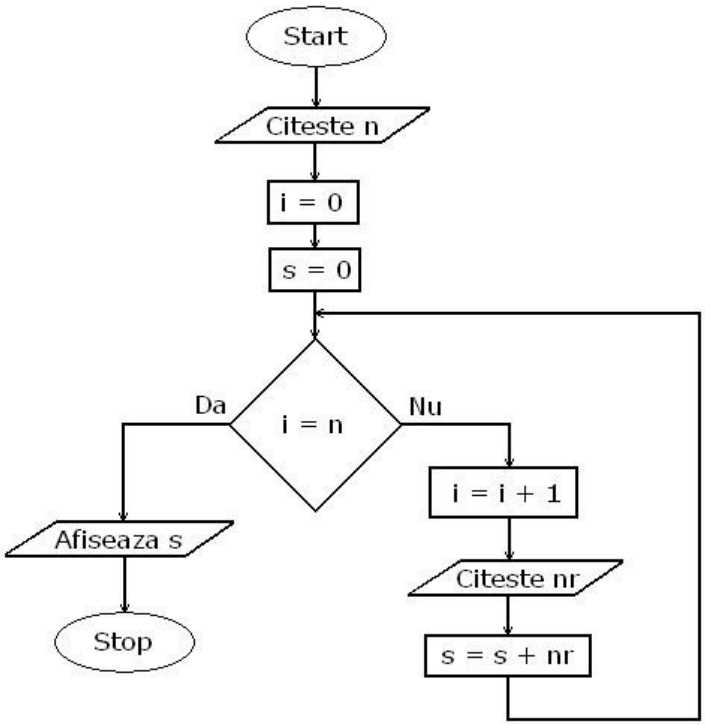
5. Să se citească două numere. Să se calculeze maximului a celor 2 numere.



6. Să se realizeze schema logică asociată cu rezolvarea unei ecuaţii de gradul I.



7. Să se realizeze schema logică asociată cu calculul sumei a n numere citite de la tastatură.



Probleme propuse

1. Să se facă schema logică pentru un program care sa citească de la tastatura numele si vârsta dvs. si sa le afișeze.
2. Să se facă schema logică pentru un program C care afișează produsul a doua variabile întregi, citite de la tastatură.
3. Să se facă schema logică pentru un program C care convertește un unghi din grade în radiani *(rad=grad*pi/180)*.
4. Să se facă schema logică pentru un program C care face conversia din grade Celsius în grade Fahrenheit, *C = (F – 32)* 5/9.*
5. Să se facă schema logică pentru un program C care afișează cifra unitarilor unei variabile de tip întreg.
6. Să se facă schema logică pentru un program care citește de la tastatura lungimile laturilor unui triunghi si afișează aria acestuia, calculata cu formula lui Heron ($A_\Delta = \sqrt{p(p-a)(p-b)(p-c)}$, unde $p = \frac{a+b+c}{2}$)

7. Se citesc de la tastatură două numere. Să se calculeze suma, produsul, diferența dintre cele două numere și să se interschimbe numerele.
8. Să se facă schema logică pentru un program C care să citească de la tastatură raza unui cerc, apoi să afişeze aria acestuia.
9. Să se facă schema logică pentru un program C care citeşte bazele şi înălţimea unui tapez, apoi să afişeze aria acestuia. (Aria $_{trapez}$= (Baza $_{mare}$ + Baza $_{mică}$)*înălțimea / 2)
10. Se citesc de la tastatură două numere intregi. Să se calculeze suma dintre restul împărţirii fiecărui număr la 2.
11. Se citesc următoarele numere 10,10, 20,30,20. Să se calculeze media aritmetică şi media aritmetică ponderată a acestora (M_P = (2*10+2*20+1*30)/(2+2+1)).
12. Se citesc de la tastatură două numere întregi. Să se calculeze media geometrică a acestora (M_G = sqrt(a*b)).
13. Să se realizeze schema logică pentru rezolvare ecuaţiei de gradul al doilea.